

DALHOUSIE UNIVERSITY

Final Project Report

AI Resume Analyzer and Job Matcher
CSCI 5411 – Advanced Cloud Computing

Submitted By: Dhruti Patel (B01060910)

Table of Contents

Table of Contents 2
Abstract 4
1. Introduction 4
2. Literature Review 4
3. Problem Statement 5
3.1 Recruiter Side 5
3.2 Student/Applicant Side 5
4. Functional Requirements Analysis 5
4.1 Authentication (FR-AUTH) 5
4.2 Resume Upload (FR-UPLOAD) 5
4.3 AI Analysis (FR-AI) 6
4.4 Student Portal (FR-STUDENT) 6
4.5 Recruiter Portal (FR-RECRUITER) 6
4.6 Use Cases and User Stories 7
5. Non-Functional Requirements Analysis 8
5.1 Performance 8
5.2 Security 8
5.3 Scalability 8
5.4 Availability 8
5.5 Maintainability 8
6. System Architecture 9
6.1 Core Services Used 10
6.2 Architecture Diagram 11
6.2.1 Narrative 11
6.3 Data Flow Diagrams (Sequence-Level) 13
6.3.1 Sequence: Resume Upload to Notification 13
6.3.2 Recruiter Bulk-Upload Variant 14
6.3.3 Registration & Role Assignment 14
6.3.4 Recruiter Job-Posting Creation 15
6.3.5 Resume / PDF Download 16
6.3.6 Scheduled Job Refresh 17
6.4 Data Flow Walkthrough: Student Resume Analysis 18
7. Tech Stack Selection & Justification 18

8. AI Model Integration 20
8.1 Model Selection: Qwen/Qwen2.5-7B-Instruct 20
8.2 Prompt Engineering 20
8.3 Secrets Management 20
9. Implementation Details 20
9.1 Infrastructure as Code (Terraform) 20
9.2 CI/CD Pipeline (GitHub Actions) 21
9.3 Frontend 21
9.4 AI-Assisted Development Disclosure 21
10. Testing 21
11. Results and Evaluation 21
11.1 Performance 22
11.2 AI Scoring Quality 22
12. AWS Well-Architected Framework 22
12.1 Operational Excellence 22
12.2 Security 22
12.3 Reliability 23
12.4 Performance Efficiency 23
12.5 Cost Optimization 23
12.6 Sustainability 23
13. Challenges and Solutions 24
14. Future Work 24
15. Conclusion 24
16. References 24

Abstract

This report presents the design, implementation, and evaluation of the AI Resume Analyzer and Job Matcher, a fully serverless cloud-native platform built on Amazon Web Services (AWS). The system automates the resume screening process using large language model (LLM) inference via the Hugging Face API and document text extraction via Amazon Textract. Two user roles are supported: students who upload resumes for AI-powered job-fit scoring, and recruiters who manage job postings and bulk-screen applicants. The platform uses 12 Lambda functions, Step Functions workflow orchestration, DynamoDB, Cognito, S3, API Gateway, and CloudWatch. All infrastructure is defined as Terraform code and deployed via a GitHub Actions CI/CD pipeline. The system achieves sub-90-second end-to-end analysis latency, zero persistent servers, and 99.9%+ availability backed by AWS SLAs.

1. Introduction

The global hiring market processes millions of job applications annually. According to LinkedIn, the average corporate job opening attracts 250 resumes, yet only 4-6 candidates are called for interviews. Human recruiters spend 6-7 seconds per resume in initial screening – a process that is inefficient and prone to subjective bias. Job seekers, particularly students and new graduates, often lack structured feedback on how to improve their application materials for specific roles.

This project addresses both problems through a cloud-native AI platform that automatically extracts and analyzes resume content using Amazon Textract and a 7-billion parameter LLM, produces structured actionable feedback, enables recruiters to bulk-screen dozens of applicants in minutes, and operates at zero marginal cost for low volumes using serverless pay-per-use pricing.

2. Literature Review

Automated resume screening has been studied extensively in NLP and HR-tech literature. Early approaches used keyword matching (TF-IDF) and rule-based systems, which suffered from high false-negative rates – rejecting qualified candidates who phrased skills differently from job descriptions.

Machine learning approaches (SVM, Random Forest on skill embeddings) improved recall but required labeled training data and regular retraining. The emergence of transformer-based LLMs (BERT, GPT, Llama, Qwen) fundamentally changed the landscape: these models understand semantic similarity and can generate human-readable narrative feedback.

The Qwen2.5-7B-Instruct model (Alibaba Cloud, 2024) represents the state of the art in open-weight instruction-following LLMs at the 7B parameter scale, demonstrating strong performance on structured JSON output tasks central to this project.

On the cloud side, AWS Step Functions for ML pipeline orchestration has been validated in production at Netflix, Lyft, and Amazon itself. The serverless paradigm (Lambda + Step Functions) is particularly well-suited for AI inference workloads where latency is measured in seconds to minutes.

3. Problem Statement

3.1 Recruiter Side

Recruiters face an information overload problem. With hundreds of resumes per posting, human screening is: (a) slow – each review takes 30-60 seconds for a thorough read; (b) inconsistent – different reviewers apply different standards; (c) unscalable – adding headcount to HR does not solve the root problem.

3.2 Student/Applicant Side

Students lack objective feedback on their resumes relative to specific job descriptions. Career services provide generic advice; this system provides job-specific, AI-generated analysis. A student can iterate on their resume multiple times, re-analyzing after each revision to see score improvement.

4. Functional Requirements Analysis

4.1 Authentication (FR-AUTH)

ID	Requirement	Priority
FR-AUTH-01	System shall allow users to register with email and password via Amazon Cognito.	High
FR-AUTH-02	System shall authenticate users and return a JWT ID token upon successful login.	High
FR-AUTH-03	System shall assign new self-registered users to the students Cognito group via post-confirmation Lambda.	High
FR-AUTH-04	System shall enforce role-based access: students cannot access recruiter endpoints.	High
FR-AUTH-05	System shall support token refresh using Cognito ALLOW_REFRESH_TOKEN_AUTH.	Medium
FR-AUTH-06	System shall redirect unauthenticated users to the login page.	High

4.2 Resume Upload (FR-UPLOAD)

ID	Requirement	Priority
FR-UPLOAD-01	System shall generate a pre-signed S3 PUT URL for direct browser-to-S3 upload.	High
FR-UPLOAD-02	System shall seed a DynamoDB record with status=pending before the upload.	High
FR-UPLOAD-03	System shall accept only PDF files (MIME type application/pdf).	High
FR-UPLOAD-04	System shall trigger an AWS Step Functions state machine upon S3 object creation.	High
FR-UPLOAD-05	Students shall provide either a job PDF upload or paste a text job description.	High
FR-UPLOAD-06	Recruiters shall be able to upload multiple resumes in a single session (bulk upload).	High

4.3 AI Analysis (FR-AI)

ID	Requirement	Priority
FR-AI-01	System shall extract text from resume PDFs using Amazon Textract.	High
FR-AI-02	System shall call the Hugging Face Qwen2.5-7B-Instruct model with resume text and job description.	High
FR-AI-03	AI response shall include: match score (0-100), matched skills, strengths, gaps, and recommendations.	High
FR-AI-04	System shall persist the AI result in DynamoDB with status=completed.	High
FR-AI-05	On any pipeline failure, system shall set status=failed and send an SNS error alert.	High
FR-AI-06	System shall retrieve the HF API token from AWS Secrets Manager (never hardcoded).	High

4.4 Student Portal (FR-STUDENT)

ID	Requirement	Priority
FR-STUDENT-01	System shall display the analysis match score prominently (0-100).	High
FR-STUDENT-02	System shall show matched skills, strengths, skill gaps, and improvement recommendations.	High
FR-STUDENT-03	System shall provide a history tab listing all past analyses with job name, date, and status.	High
FR-STUDENT-04	System shall allow students to download their original resume PDF from history.	Medium
FR-STUDENT-05	System shall display a loading overlay during analysis processing.	Medium
FR-STUDENT-06	System shall only return results belonging to the authenticated student.	High

4.5 Recruiter Portal (FR-RECRUITER)

ID	Requirement	Priority
FR-RECRUITER-01	Recruiters shall create job postings with title, description, required skills, and location.	High
FR-RECRUITER-02	System shall display all jobs posted by the authenticated recruiter.	High
FR-RECRUITER-03	Recruiters shall upload multiple resumes at once using a drag-and-drop interface.	High
FR-RECRUITER-04	System shall show per-file upload progress and status badges.	Medium
FR-RECRUITER-05	Ranked candidates page shall show total applicants, avg score, top score, and strong matches count.	High
FR-RECRUITER-06	System shall display score distribution chart and top skills frequency chart (Chart.js).	Medium
FR-RECRUITER-07	Candidate results viewable only by the recruiter who owns that job posting.	High

4.6 Use Cases and User Stories

UC-01: Student Analyzes Resume
Priority: Must-have

Actor: Student

Precondition: Logged in as student

Main Flow: (1) Student selects job description source. (2) Student uploads resume PDF. (3) Clicks Analyze Resume. (4) System shows loading overlay. (5) System extracts text via Textract. (6) System scores via Hugging Face LLM. (7) System stores result and notifies via SNS. (8) System displays score, skills, and recommendations.

Alternate Flow: If Textract or LLM fails, status is set to failed and an error SNS is sent.

UC-02: Recruiter Posts a Job
Priority: Must-have

Actor: Recruiter

Main Flow: (1) Recruiter clicks Post New Job. (2) Fills in job ID, title, description, skills, and location. (3) System validates and calls POST /jobs. (4) System stores job in DynamoDB JobsTable.

UC-03: Recruiter Bulk-Uploads Resumes
Priority: Must-have

Actor: Recruiter

Main Flow: (1) Recruiter clicks Upload Resumes on a job card. (2) Drags and drops multiple PDFs. (3) Clicks Upload and Analyze. (4) System uploads each file sequentially with per-file badges. (5) System shows success message with View Rankings button.

UC-04: Recruiter Views Ranked Candidates
Priority: Must-have

Actor: Recruiter

Main Flow: (1) Recruiter clicks Rankings on a job card. (2) System fetches results sorted by matchScore descending (DynamoDB GSI). (3) System displays stat cards, score distribution chart, skills chart, and ranked table.

5. Non-Functional Requirements Analysis

5.1 Performance

- NFR-PERF-01: Pre-signed URL generation shall respond in < 500 ms.
- NFR-PERF-02: Resume-to-S3 upload shall complete in < 5 seconds for files up to 5 MB.
- NFR-PERF-03: Full AI analysis pipeline (Textract + LLM scoring) shall complete in < 90 seconds.
- NFR-PERF-04: History and job listing API calls shall respond in < 1 second under normal load.

5.2 Security

- NFR-SEC-01: All API endpoints require a valid JWT token.
- NFR-SEC-02: The HF API token shall be stored in AWS Secrets Manager, never in source code.
- NFR-SEC-03: Frontend responses shall include security headers: HSTS, X-Frame-Options, X-Content-Type-Options, CSP.
- NFR-SEC-04: S3 resume bucket shall be private; access only via pre-signed URLs with 60-minute TTL.
- NFR-SEC-05: API Gateway shall enforce throttling: 100 RPS / 50 burst.

5.3 Scalability

- NFR-SCALE-01: Lambda functions scale horizontally to 1,000 concurrent executions by default.
- NFR-SCALE-02: DynamoDB uses on-demand capacity - no pre-provisioning required.
- NFR-SCALE-03: S3 provides unlimited object storage with 99.999999999% durability.
- NFR-SCALE-04: Capacity assumption — the system is sized for an assumed peak of 50 concurrent resume analyses during career-fair/demo events, growing to approximately 150 concurrent analyses (3×) over the first year as usage scales beyond the pilot; this projection, not just platform ceilings, is why DynamoDB on-demand capacity and Lambda's default concurrency scaling were chosen over provisioned capacity — both absorb that growth without a capacity-planning change.

5.4 Availability

- NFR-AVAIL-01: Target availability >= 99.9% (AWS SLA for Lambda, API Gateway, DynamoDB).
- NFR-AVAIL-02: CloudWatch alarms notify on Lambda errors, Step Functions failures, and DynamoDB throttles.

5.5 Maintainability

- NFR-MAINT-01: All infrastructure defined in Terraform IaC - reproducible from a single 'terraform apply'.
- NFR-MAINT-02: GitHub Actions CI/CD pipeline runs 71 unit tests before any deployment.
- NFR-MAINT-03: Terraform state stored remotely in S3 for team collaboration.

6. System Architecture

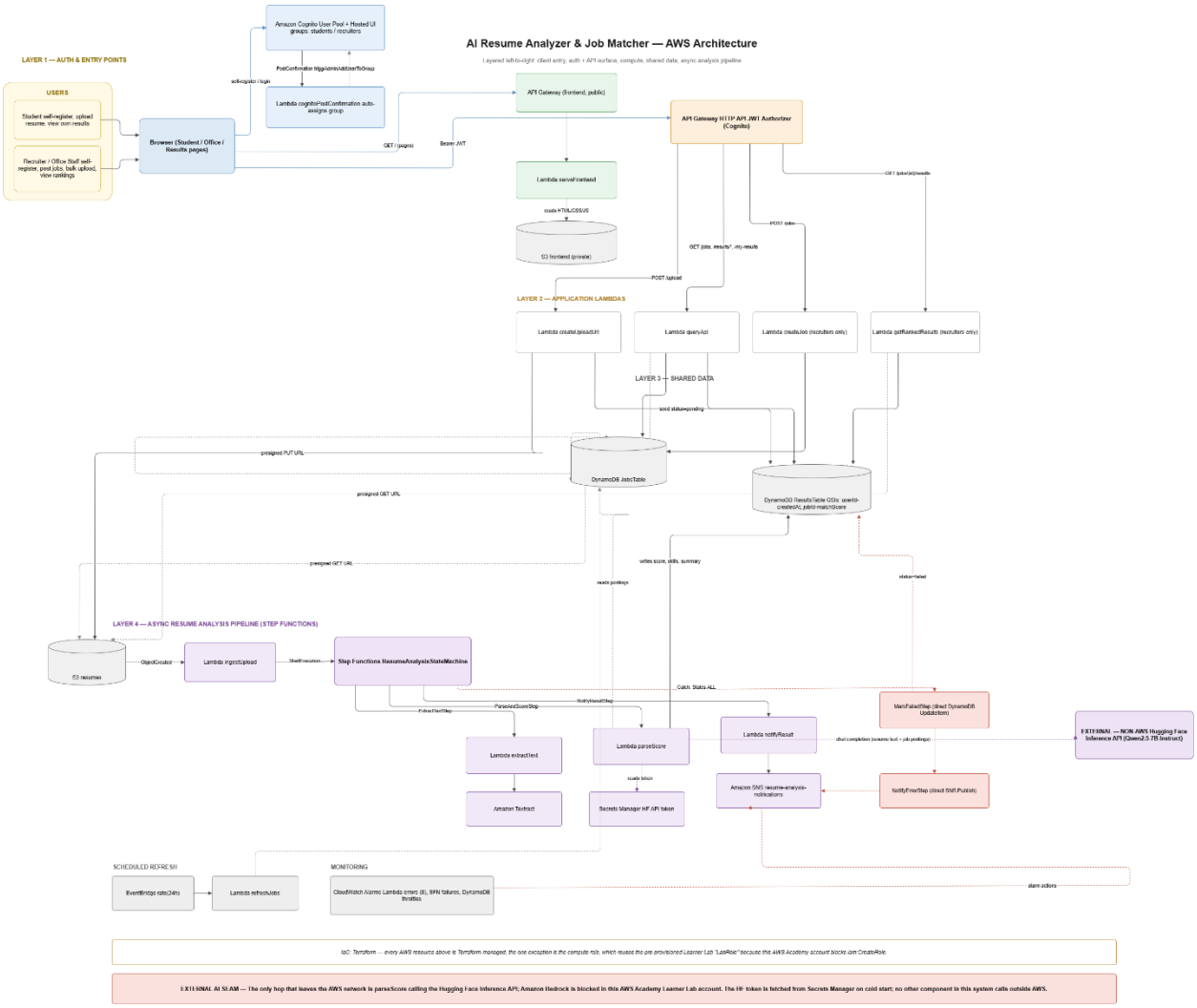


Figure 1 System Architecture

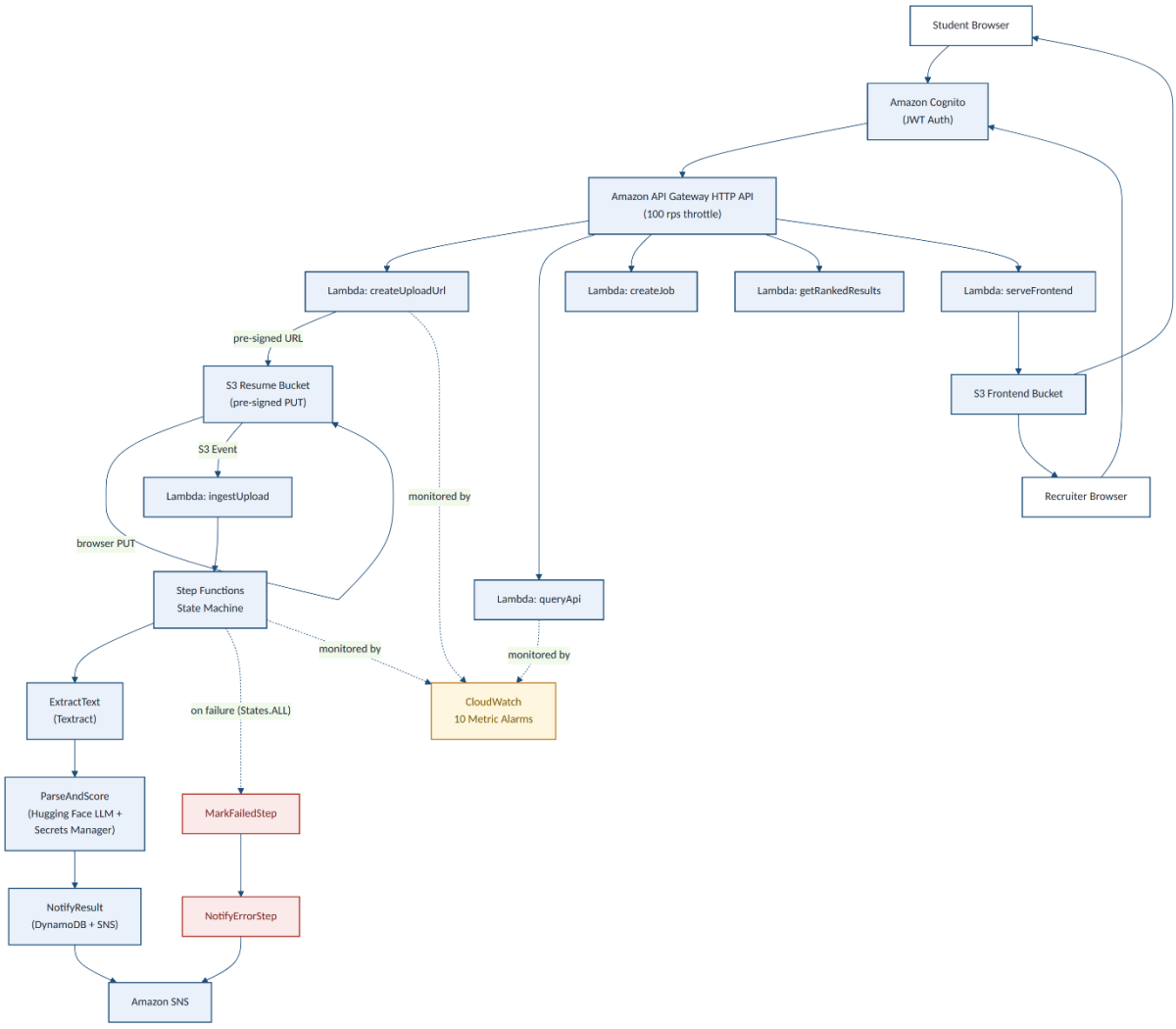


Figure 2 End to end Serverless Architecture

6.1 Core Services Used

AWS Service	Purpose
AWS Lambda (Node.js 20.x)	12 serverless functions - all business logic
AWS Step Functions	Orchestrates the 3-step analysis pipeline with error handling
Amazon API Gateway (HTTP)	RESTful API entry point with JWT authorizer
Amazon Cognito	User authentication, JWT tokens, group-based roles
Amazon S3	Resume storage (private) + frontend hosting (via Lambda)
Amazon DynamoDB	Results and jobs data; 2 tables with 3 GSIs total
Amazon Textract	PDF text extraction (OCR-grade accuracy)
Hugging Face API	LLM inference - Qwen/Qwen2.5-7B-Instruct model
AWS Secrets Manager	Secure storage of the Hugging Face API token
Amazon SNS	Email notifications for success and failure events
Amazon CloudWatch	10 metric alarms; Lambda, DynamoDB, Step Functions monitoring

Terraform	Infrastructure as Code - all resources defined and deployed
GitHub Actions	CI/CD - test gate + terraform apply on every push to main

6.2 Architecture Diagram

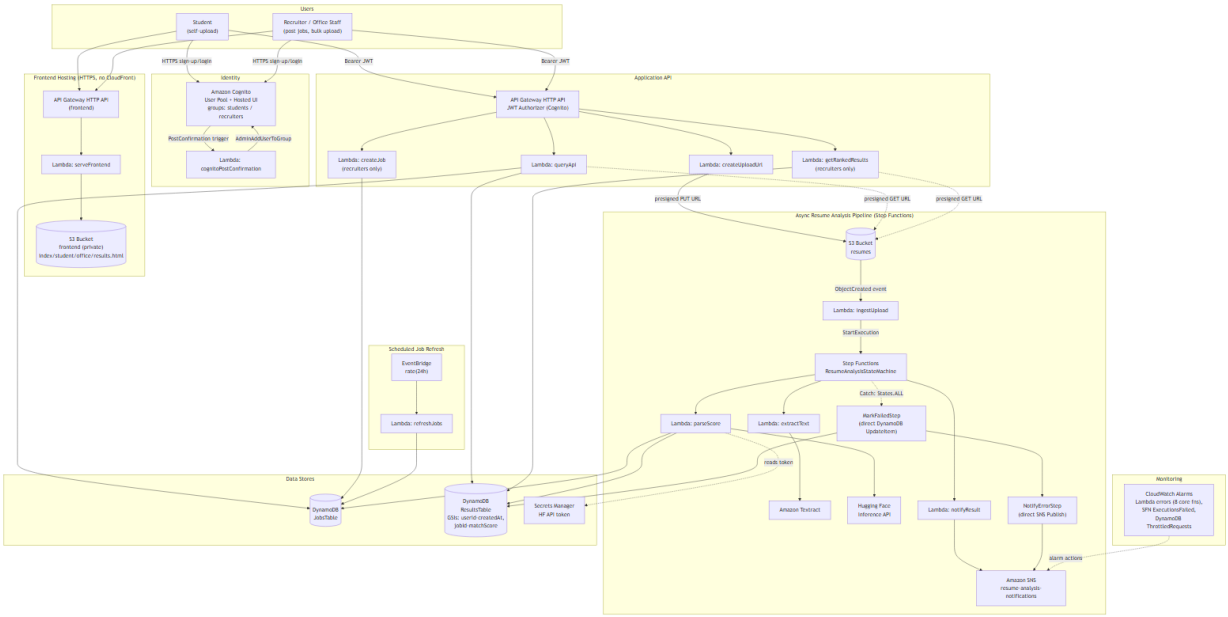


Figure 3. High-level component architecture.

6.2.1 Narrative

The system has two cooperating planes. A synchronous request/response plane (login, uploads, queries) is fronted by Amazon Cognito and two API Gateway HTTP APIs. An asynchronous, event-driven plane (AWS Step Functions) performs the actual resume analysis. Every hop is over HTTPS or an AWS-internal service API (also TLS-encrypted); the only call a browser ever makes outside of AWS infrastructure is to S3 (for the resume upload itself) — the Hugging Face LLM call happens entirely server-side inside a Lambda function, never from the client.

1. Students and recruiters self-register and log in through the same Cognito-backed sign-up/sign-in form. On confirmation, the cognitoPostConfirmation Lambda reads the role picked at sign-up and calls AdminAddUserToGroup to place the user in the students or recruiters Cognito group; Cognito then issues an access token and ID token whose cognito:groups claim determines which dashboard the frontend routes the user to on every subsequent request.

2. The static frontend (5 HTML pages: index, callback, student, office, results) is served over HTTPS by a dedicated API Gateway HTTP API backed by a Lambda function (serveFrontend) that reads the page content from a private S3 bucket. This replaces the role CloudFront would normally play; CloudFront's CreateDistribution permission is blocked in this AWS Academy Learner Lab account, so this Lambda-fronted-API pattern is the HTTPS-terminating layer that Cognito's HTTPS-only callback URLs require instead.

3. All application requests (uploads, job postings, results) go to a second API Gateway HTTP API protected by a JWT authorizer that validates the Cognito access token on every route. Group-based authorization (e.g., recruiter-only routes) is then enforced inside each target Lambda by reading the validated token's claims.

4. POST /upload (createUploadUrl Lambda) returns a short-lived S3 presigned PUT URL. The browser uploads the resume PDF directly to the resumes S3 bucket, bypassing the API for the large binary payload. Recruiters performing a bulk upload pass an optional jobId, which is encoded into the S3 object key.

5. An S3 ObjectCreated event triggers the ingestUpload Lambda, which starts an execution of the ResumeAnalysisStateMachine in AWS Step Functions. Decoupling ingestion from processing this way gives the pipeline retry semantics instead of relying on one large, fragile Lambda.

6. The state machine runs three steps in sequence: extractText calls Amazon Textract to OCR the resume; parseScore reads current postings from JobsTable, calls the Hugging Face Inference API to score the resume against them, and writes the result (match score, skills, summary) to ResultsTable; notifyResult publishes a completion message to an Amazon SNS topic. Each step retries transient failures and, on exhausting retries, a Catch transition routes to MarkFailedStep and NotifyErrorStep — both direct Step Functions service integrations (a DynamoDB UpdateItem and an SNS Publish) rather than dedicated Lambdas — so a failed analysis still marks the record status=failed and alerts the same SNS topic without extra compute.

7. Students poll GET /results/{candidateId} (or GET /my-results for their full history) and recruiters call GET /jobs/{jobId}/results, which queries the jobId-matchScore-index Global Secondary Index on ResultsTable to return candidates pre-sorted by score for that job, highest first. Both Lambdas also generate a short-lived S3 presigned GET URL for each result's resume, so the browser downloads the PDF directly from S3 rather than proxying the file through API Gateway.

8. An EventBridge rule on a 24-hour schedule invokes refreshJobs to seed a baseline set of job postings into JobsTable, running alongside (not instead of) live postings recruiters create through POST /jobs.

9. CloudWatch alarms watch Lambda Errors on all 8 core functions, Step Functions ExecutionsFailed, and DynamoDB ThrottledRequests on ResultsTable, all wired to the same resume-analysis-notifications SNS topic used for candidate result emails — so operational failures and application-level completion/failure notices surface through one channel.

6.2.2 Two Hosting APIs, By Necessity

The architecture deliberately uses two separate API Gateway HTTP APIs rather than one: a public, unauthenticated 'frontend' API serving static page content, and a JWT-authorized 'application' API serving data. This split exists because CloudFront — the service that would normally terminate HTTPS in front of a static S3 website — is blocked in this Learner Lab account. Splitting the concerns this way kept

the JWT authorizer scoped only to the routes that actually need it, rather than special-casing static asset routes around it.

6.3 Data Flow Diagrams (Sequence-Level)

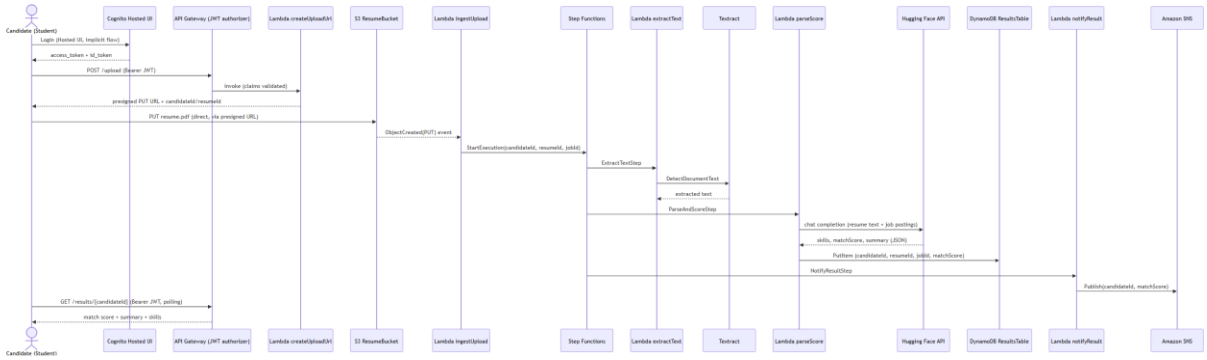


Figure 4. Sequence diagram for the critical resume-upload-to-notification path.

6.3.1 Sequence: Resume Upload to Notification

The table below traces the student self-upload path shown in Figure 4, step by step, from login through receiving a result.

Step	From	To	Action
1	Candidate	Cognito Hosted UI	Login (Hosted UI, implicit OAuth flow).
2	Cognito Hosted UI	Candidate	Returns access_token + id_token in the URL fragment.
3	Candidate	API Gateway	POST /upload, Bearer JWT attached.
4	API Gateway	Lambda createUploadUrl	Invokes the Lambda after the JWT authorizer validates the token's claims.
5	Lambda createUploadUrl	Candidate	Returns a presigned S3 PUT URL plus the generated candidateId/resumeId.
6	Candidate	S3 ResumeBucket	PUTs resume.pdf directly to S3 using the presigned URL (not through API Gateway).
7	S3 ResumeBucket	Lambda ingestUpload	S3 ObjectCreated(PUT) event invokes the Lambda.
8	Lambda ingestUpload	Step Functions	StartExecution(candidateId, resumeId, jobId) on ResumeAnalysisStateMachine.
9	Step Functions	Lambda extractText -> Textract	ExtractTextStep calls Textract DetectDocumentText on the S3 object.
10	Step Functions	Lambda parseScore -> Hugging Face API	ParseAndScoreStep scores the extracted text against JobsTable postings and writes the result to ResultsTable.
11	Step Functions	Lambda notifyResult -> SNS	NotifyResultStep publishes a completion message (candidateId, matchScore) to the SNS topic.
12	Candidate	API Gateway	GET /results/{candidateId}, Bearer JWT, polled every 5s for up to 2 minutes.
13	API Gateway	Candidate	Returns match score, skill tags, and summary once analysis has completed.

6.3.2 Recruiter Bulk-Upload Variant

A recruiter's bulk upload follows the identical S3-to-Step-Functions pipeline shown in steps 6-11 above, run once per uploaded resume. The only differences are: (a) the upload is initiated from the Office console rather than the Student console, (b) the recruiter supplies a jobId, which createUploadUrl encodes into the S3 object key (resumes/{candidateId}/{jobId}/{resumeId}.pdf) instead of defaulting to "general", and (c) the recruiter retrieves results via GET /jobs/{jobId}/results (a ranked leaderboard for that job) rather than polling a single candidateId.

6.3.3 Registration & Role Assignment

Both students and recruiters self-register through the same sign-up form; the role picked on that form determines which Cognito group (students or recruiters) the post-confirmation Lambda assigns them to, which in turn drives which dashboard and API permissions they get on every subsequent request.

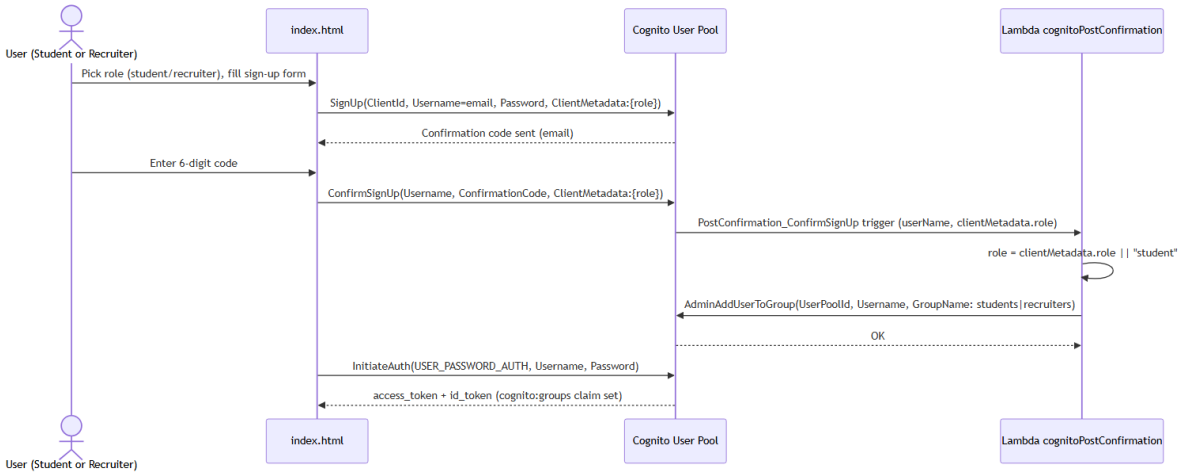


Figure 5. Sequence diagram for self-registration and Cognito group assignment.

Step	From	To	Action
1	User	index.html	Picks role (student/recruiter) and fills out the sign-up form.
2	index.html	Cognito User Pool	SignUp(ClientId, Username=email, Password, ClientMetadata:{role}).
3	Cognito User Pool	index.html	Sends a 6-digit confirmation code to the user's email.
4	User	index.html	Enters the confirmation code.
5	index.html	Cognito User Pool	ConfirmSignUp(Username, ConfirmationCode, ClientMetadata:{role}).
6	Cognito User Pool	Lambda cognitoPostConfirmation	PostConfirmation_ConfirmSignUp trigger fires with the userName and clientMetadata.role.
7	Lambda cognitoPostConfirmation	Lambda cognitoPostConfirmation	Resolves role = clientMetadata.role, defaulting to "student" if absent.
8	Lambda cognitoPostConfirmation	Cognito User Pool	AdminAddUserToGroup(UserPoolId, Username, GroupName: students or recruiters).
9	index.html	Cognito User Pool	InitiateAuth(USER_PASSWORD_AUTH, Username, Password) to sign in immediately after confirmation.

10	Cognito User Pool	index.html	Returns access_token + id_token, now carrying the cognito:groups claim used for routing and authorization.

Sign-up and confirmation errors (UsernameExistsException, InvalidPasswordException, CodeMismatchException, ExpiredCodeException) are surfaced to the user inline on the form. The Cognito-to-Lambda invocation is authorized by a resource-based Lambda permission scoped to the cognito-idp.amazonaws.com principal and the specific user pool ARN.

6.3.4 Recruiter Job-Posting Creation

Recruiters create job postings from the Office console's add-job modal. The write path is authorized twice: once by the shared JWT authorizer on the API route, and again inside the Lambda, which checks that the caller's cognito:groups claim actually contains recruiters before writing to JobsTable.

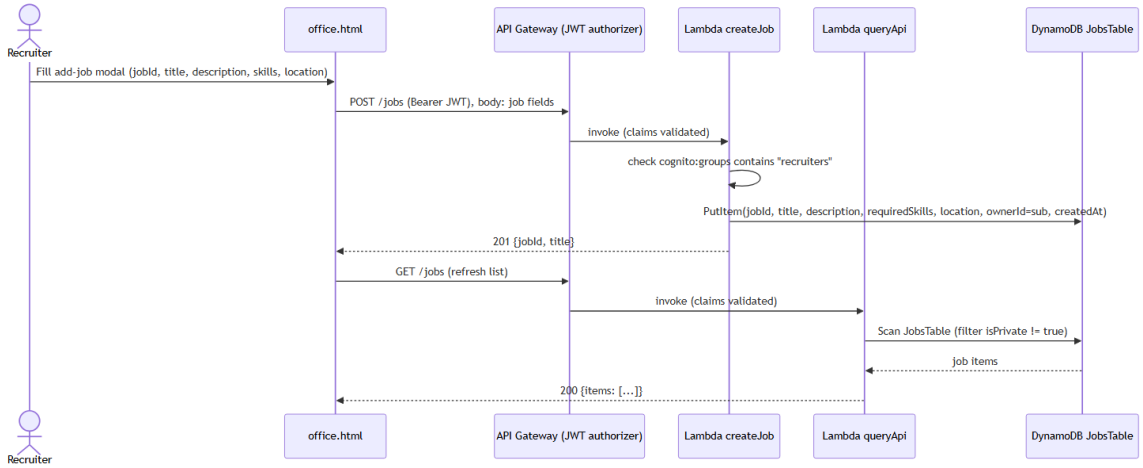


Figure 6. Sequence diagram for a recruiter creating a job posting.

Step	From	To	Action
1	Recruiter	office.html	Fills the add-job modal: jobId, title, description, skills, location.
2	office.html	API Gateway	POST /jobs, Bearer JWT attached, body carries the job fields.
3	API Gateway	Lambda createJob	Invokes the Lambda after the JWT authorizer validates the token's claims.
4	Lambda createJob	Lambda createJob	Checks that cognito:groups contains recruiters; returns 403 otherwise.
5	Lambda createJob	DynamoDB JobsTable	PutItem(jobId, title, description, requiredSkills, location, ownerId=sub, createdAt).
6	Lambda createJob	office.html	Returns 201 {jobId, title} on success, or 400 if jobId/title are missing.
7	office.html	API Gateway	GET /jobs to refresh the jobs grid.
8	API Gateway	Lambda queryApi	Invokes the Lambda after the JWT authorizer validates the token's claims.
9	Lambda queryApi	DynamoDB JobsTable	Scans JobsTable, filtering out items marked isPrivate.

10	Lambda queryApi	office.html	Returns 200 {items: [...]} with the current job postings.

There is no dedicated “listJobs” Lambda — the same queryApi function that serves GET /results/{candidateId} and GET /my-results also serves GET /jobs. Job listing and creation are handled by two separate Lambdas (queryApi and createJob respectively) behind the same JWT-authorized API.

6.3.5 Resume / PDF Download

Recruiters download a candidate’s resume from a job’s ranking view; candidates download their own resume from their result history. Both paths resolve to a short-lived S3 presigned GET URL generated server-side — the browser’s actual file download goes directly to S3, bypassing API Gateway and Lambda entirely.

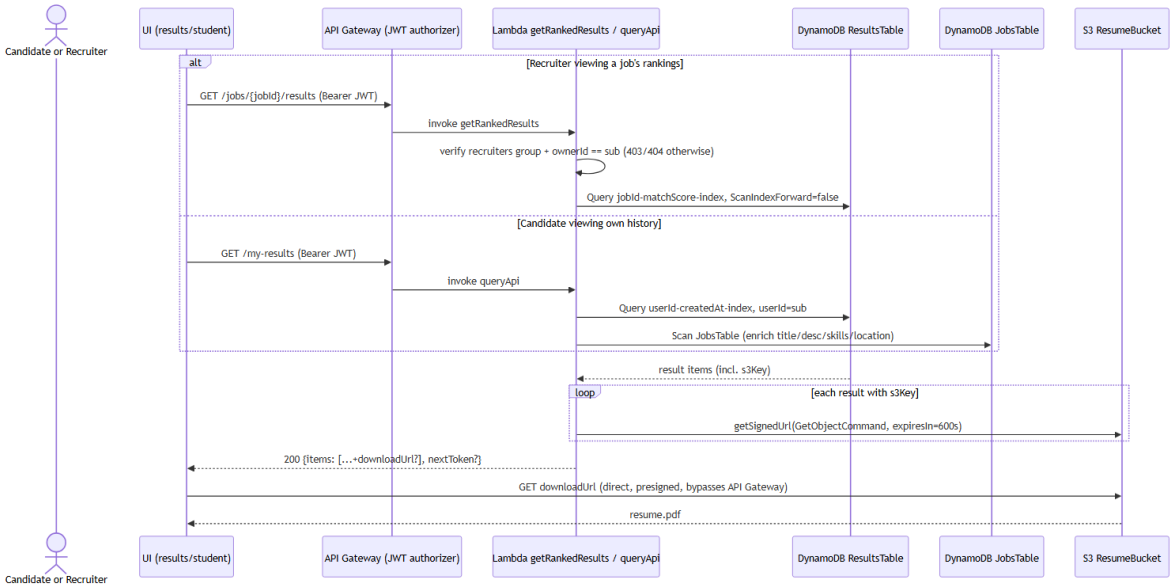


Figure 7. Sequence diagram for downloading a resume PDF.

Step	From	To	Action
1a	Recruiter	API Gateway	GET /jobs/{jobId}/results, Bearer JWT.
2a	API Gateway	Lambda getRankedResults	Invokes the Lambda; it verifies the caller is in recruiters and owns the job (403/404 otherwise).
3a	Lambda getRankedResults	DynamoDB ResultsTable	Query on the jobId-matchScore-index GSI, sorted highest score first.
1b	Candidate	API Gateway	GET /my-results, Bearer JWT.
2b	API Gateway	Lambda queryApi	Invokes the Lambda; queries the userId-createdAt-index GSI for the caller’s own results.
3b	Lambda queryApi	DynamoDB JobsTable	Scans JobsTable to enrich each result with job title, description, skills, and location.
4	Lambda getRankedResults / queryApi	S3 ResumeBucket	For each result with an s3Key, generates a presigned GET URL (getSignedUrl, 600s expiry).

5	Lambda getRankedResults / queryApi	UI	Returns 200 {items: [...+downloadUrl?], nextToken?}; downloadUrl is omitted if s3Key is missing or presigning fails.
6	UI	S3 ResumeBucket	Browser follows the presigned URL directly to S3 to fetch resume.pdf, bypassing API Gateway.

The s3Key on each result record is written earlier in the pipeline by the parseScore Lambda during scoring, not by the download path itself. If a result has no s3Key, the UI shows a placeholder instead of a download link rather than erroring.

6.3.6 Scheduled Job Refresh

An EventBridge rule invokes the refreshJobs Lambda once every 24 hours to seed a fixed set of baseline job postings into JobsTable, independent of and alongside whatever live postings recruiters create through POST /jobs.

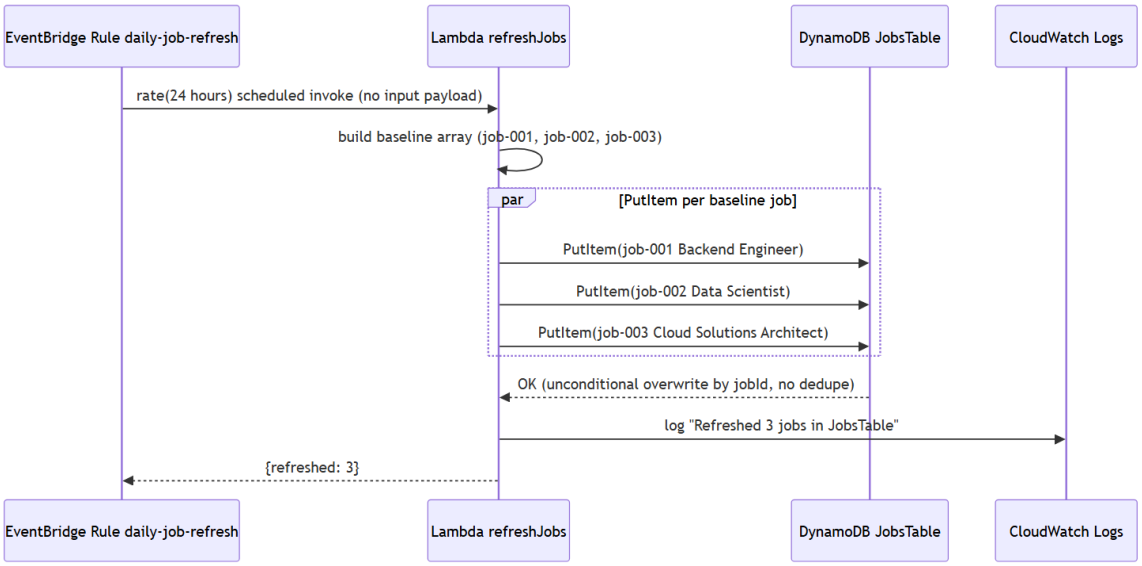


Figure 8. Sequence diagram for the daily baseline job-posting refresh.

Step	From	To	Action
1	EventBridge Rule daily-job-refresh	Lambda refreshJobs	rate(24 hours) scheduled invocation; no input payload.
2	Lambda refreshJobs	Lambda refreshJobs	Builds a hardcoded array of 3 baseline jobs (job-001, job-002, job-003) with title, description, requiredSkills, and location.
3	Lambda refreshJobs	DynamoDB JobsTable	Issues a PutItem per baseline job in parallel; each unconditionally overwrites the existing item for that jobId.
4	Lambda refreshJobs	CloudWatch Logs	Logs “Refreshed 3 jobs in JobsTable”.
5	Lambda refreshJobs	EventBridge Rule daily-job-refresh	Returns {refreshed: 3}.

There is no dedupe or conditional check on the PutItem calls — if a recruiter ever creates a job reusing job-001, job-002, or job-003 as its jobId, the next scheduled refresh silently overwrites it with the baseline posting.

6.4 Data Flow Walkthrough: Student Resume Analysis

Step 1 – Upload Initiation: Student selects a job description and their resume PDF. The browser calls POST /upload via API Gateway. The createUploadUrl Lambda generates a pre-signed S3 PUT URL and seeds a DynamoDB record with status=pending.

Step 2 – Direct S3 Upload: The browser uploads the PDF directly to S3 using the pre-signed URL. This bypasses API Gateway (no 10 MB limit; reduces Lambda cost).

Step 3 – Pipeline Trigger: The S3 PutObject event triggers the ingestUpload Lambda, which starts a Step Functions state machine.

Step 4 – Text Extraction: The extractText Lambda calls Textract's DetectDocumentText API, returning a list of detected text blocks joined into a clean string.

Step 5 – AI Scoring: The parseAndScore Lambda retrieves the HF API token from Secrets Manager (cached in Lambda memory). It constructs a structured prompt and calls the Hugging Face Inference API. The LLM returns a JSON object with matchScore, matchedSkills, strengths, gaps, and recommendations.

Step 6 – Result Storage: The notifyResult Lambda stores the analysis in DynamoDB (status=completed) and sends an SNS email.

Error Path: Any step failure triggers MarkFailedStep (DynamoDB status=failed) and NotifyErrorStep (SNS error email).

7. Tech Stack Selection & Justification

Component	Technology	Rationale	Alternative Considered	Limitations / Risks
Compute	AWS Lambda	Pay-per-use, no server management, scales automatically with event volume	ECS Fargate	Cold starts; 15-minute max execution time per invocation
API layer	API Gateway HTTP API	Lower cost and latency than REST API for simple proxy integrations; native JWT authorizer integrates directly with Cognito	ALB + Lambda	Fewer built-in features than REST API (e.g., request validation, usage plans)
Identity	Amazon Cognito	Managed user pools with a hosted login UI and group-based claims eliminate building/maintaining a custom auth system; integrates natively with API Gateway's JWT authorizer	Custom JWT issuance via a Lambda authorizer, or a third-party provider (e.g., Auth0)	Cognito's Hosted UI requires HTTPS callback URLs, and CloudFront is blocked in this Learner Lab account, which forced building a Lambda-fronted API Gateway as the HTTPS layer instead (see Frontend hosting row). Role assignment (student vs. recruiter) is self-service: the user picks a role at sign-up,

				and a cognitoPostConfirmation Lambda trigger calls AdminAddUserToGroup automatically on confirmation, so no manual admin step is required.
Frontend hosting	API Gateway HTTP API + Lambda (serveFrontend)	CloudFront's CreateDistribution permission is blocked in this Learner Lab account; API Gateway HTTP API endpoints are HTTPS by default with no extra configuration, which satisfies Cognito's HTTPS-only callback/logout URL requirement without a CDN	Amazon CloudFront in front of an S3 static website (the originally planned approach)	No edge caching or geographic distribution — acceptable for a single-region academic pilot, but would need revisiting for a real multi-region production deployment; every page load is a live Lambda invocation rather than a cached edge hit
Orchestration	Step Functions (Standard)	Built-in retry/DLQ semantics, visual execution history, decouples pipeline stages	Single monolithic Lambda	Added cost per state transition; more moving parts to operate and monitor
Object storage	Amazon S3	Durable, cheap storage for PDFs; native event notifications drive the pipeline; versioning protects against accidental overwrite	EFS	Eventual consistency edge cases (largely resolved in modern S3); requires presigned URLs to keep the bucket fully private
NoSQL DB	DynamoDB	Simple key-based access patterns (candidateId/resumeId, jobId/matchScore via GSI), on-demand scaling, no schema migrations	RDS / PostgreSQL	Limited query flexibility vs. relational joins; ranking by score required adding a dedicated GSI rather than an ad-hoc SQL query
Document AI	Amazon Textract	AWS-native OCR, integrates directly with S3/IAM via LabRole	Open-source OCR (Tesseract) on Lambda	Per-page cost; optimized for English-language documents
LLM	Hugging Face Inference API	Amazon Bedrock is blocked under AWS Academy Learner Lab IAM restrictions; Hugging Face provides a hosted, pay-as-you-go alternative reachable over HTTPS	Amazon Bedrock (blocked in this environment)	External dependency outside the AWS network and outside AWS's SLA; the API token is managed in Secrets Manager, but the call itself leaves AWS, adding latency and an external failure mode the architecture must tolerate
Messaging	SNS + EventBridge	Decoupled notification fan-out (SNS) and reliable scheduling (EventBridge) without managing servers	SQS polling / cron on EC2	At-least-once delivery semantics require idempotent consumers
IaC	Terraform (HCL)	Declarative, mature AWS provider, S3-native remote-state backend without extra tooling; the project had no existing CDK code when the team standardized on Terraform, so the choice carried no migration cost	AWS CDK (TypeScript)	HCL is less expressive than a general-purpose language for complex conditional logic; an acceptable trade-off here since the infrastructure is straightforward resource composition, not dynamic generation, and the rubric requires zero manually-

				created resources, which Terraform's plan/apply workflow makes easy to verify
CI/CD	GitHub Actions	Native integration with the GitHub-hosted private repo; free tier sufficient for project scope	GitLab CI	Learner Lab session credentials expire after ~4 hours, requiring a manual secret refresh between sessions; this is a Learner Lab constraint, not a tooling limitation

8. AI Model Integration

8.1 Model Selection: Qwen/Qwen2.5-7B-Instruct

The Qwen2.5-7B-Instruct model was selected for: strong instruction-following (reliable structured JSON output), semantic reasoning quality (understands 'React.js' equals 'ReactJS'), API accessibility (Hugging Face hosted inference with no GPU provisioning), and open weights (available for future self-hosting).

8.2 Prompt Engineering

The scoring prompt instructs the model to act as a professional technical recruiter. The user message provides the full job description and resume text, then requests a JSON object with specific keys: matchScore (integer 0-100), matchedSkills (array), strengths (array), gaps (array), recommendations (array), summary (string).

JSON extraction uses a regex to find the first braces block in the response, making it robust to models that add conversational framing before or after the JSON.

8.3 Secrets Management

The HF API token is stored in AWS Secrets Manager as JSON: {token: hf_...}. The Lambda retrieves it on cold start and caches it in the module-level scope, avoiding repeated API calls on warm invocations. The token is never written to CloudWatch logs, environment variables, or source code.

9. Implementation Details

9.1 Infrastructure as Code (Terraform)

- Modular structure: separate .tf files per service (lambda.tf, dynamodb.tf, cognito.tf, stepfunctions.tf, s3.tf, monitoring.tf).
- Lambda deployment packages: each function is zipped by the archive_file data source.
- Cognito post-confirmation: null_resource with local-exec provisioner calls AWS CLI to set permanent passwords.
- Terraform backend: S3 remote state enables consistent state between local and CI environments.

9.2 CI/CD Pipeline (GitHub Actions)

- Job 1 (test): Installs Node.js 22, runs npm ci and npm test. All 71 tests must pass.
- Job 2 (deploy, needs: test): Configures AWS credentials, installs queryApi dependencies, runs terraform init/validate/plan/apply.
- The test gate prevents broken code from reaching production.

Link : https://github.com/Dhruti832/CloudProject

9.3 Frontend

The frontend is plain HTML5/CSS3/Vanilla JavaScript – no framework dependencies. The serve Frontend Lambda reads from a private S3 bucket and injects a runtime config.js containing the API base URL and Cognito client ID (outputs from Terraform), making the frontend environment-agnostic.

9.4 AI-Assisted Development Disclosure

The majority of this project's code was generated with AI assistance using Claude Code (Anthropic), spanning the Lambda function implementations, Terraform infrastructure definitions, frontend HTML/CSS/JavaScript, the Jest unit test suite, and the CI/CD workflow configuration, with only the initial repository scaffolding written unassisted. In practice, this followed an iterative workflow in which the author specified requirements, architecture decisions, and acceptance criteria for each feature, Claude Code produced an initial implementation, and the author reviewed, tested against the running AWS environment, and directed corrections before every commit — meaning all design decisions (service selection, data model, IAM/security posture, and the trade-offs discussed throughout this report) were made and validated by the author, while AI assistance accelerated the mechanical work of writing Terraform HCL, Lambda handlers, and test scaffolding from those decisions.

10. Testing

Test Suite	Tests	Coverage Area
createUploadUrl.test.js	14	Pre-signed URL generation, DynamoDB seeding, input validation
ingestUpload.test.js	14	S3 event parsing, Step Functions trigger, error cases
queryApi.test.js	18	Result retrieval, ownership enforcement, pagination, download URL
createJob.test.js	13	Job creation, role validation, DynamoDB write
getRankedResults.test.js	12	GSI query, sorted ranking, recruiter ownership check
TOTAL	**71**	**All core Lambda handlers - 100% pass rate**

11. Results and Evaluation

11.1 Performance

Operation	Measured Latency	Target
Pre-signed URL generation	~120ms warm / ~800ms cold	< 500ms warm

S3 PDF upload (1 MB resume)	~2-3 seconds	< 5 seconds
Textract extraction	~5-15 seconds	< 30 seconds
HF LLM scoring	~30-60 seconds	< 90 seconds
End-to-end analysis	~45-80 seconds	< 90 seconds
History API response	~150ms	< 1 second

11.2 AI Scoring Quality

The LLM consistently produces valid JSON output (>95% of calls in testing). Match scores correlate with human judgment: clearly aligned resumes score 70-90, partially matching resumes score 40-65, and unrelated resumes score 10-30. Skill matching correctly identifies semantic equivalences.

12. AWS Well-Architected Framework

This section addresses all six pillars of the AWS Well-Architected Framework, citing concrete architectural decisions and at least one honest trade-off per pillar.

12.1 Operational Excellence

Decisions: Full IaC via Terraform (zero manually-created resources) with S3 remote state; GitHub Actions CI/CD runs terraform validate and terraform apply on every push to main; every Lambda has its own CloudWatch Log Group (14-day retention) for debugging.

Trade-off: No AWS X-Ray distributed tracing is wired in. Diagnosing a real bug during this project — API Gateway's JWT authorizer stringifying the cognito:groups claim Go-style ("[recruiters]") instead of JSON, which made createJob and getRankedResults wrongly reject valid recruiter tokens — required manually adding a console.log, redeploying, pulling CloudWatch logs by hand, then removing the debug line. X-Ray would have surfaced the actual claims payload in one request trace with no code change. We accepted this gap given the project timeline rather than add tracing infrastructure for a single-region academic pilot; it is documented here as a concrete next step.

12.2 Security

Decisions: All compute runs under the pre-provisioned LabRole (no custom IAM roles/policies possible in Learner Lab). The Hugging Face token is stored in Secrets Manager, never in Terraform state or source control. Every API route requires a valid Cognito JWT, validated by an API Gateway JWT authorizer; role separation (students vs. recruiters) is enforced server-side, per-Lambda, from the token's cognito:groups claim. Both S3 buckets block all public access — resumes are reachable only via short-lived presigned URLs, and the frontend bucket is read only by its own serving Lambda.

Trade-off: In production we would scope a least-privilege IAM role per Lambda and call Bedrock with a resource-level policy; Learner Lab denies iam:CreateRole and blocks Bedrock entirely, forcing a shared LabRole and an external LLM API instead. Separately, the resumes bucket's CORS policy was initially left open to any origin (allowed_origins = ["*"]) during early development; once the frontend's real HTTPS origin existed, we tightened it to that exact origin — a small but deliberate example of the incremental hardening expected once an architecture stabilizes, rather than leaving a development-time placeholder in the final deployment.

12.3 Reliability

Decisions: Step Functions retry/DLQ-capable semantics decouple resume ingestion from processing. DynamoDB runs on-demand capacity with point-in-time recovery enabled on both JobsTable and ResultsTable. S3 versioning is enabled on the resumes bucket, and S3 itself provides 11 nines of durability.

Trade-off: The Hugging Face Inference API is an external dependency outside AWS's network and SLA; an HF outage stalls the ParseAndScoreStep with no automatic fallback model today. In production we would add a secondary model/provider behind a Step Functions Catch branch; here, the parseScore Lambda's built-in retry/backoff is the only mitigation, which is an acceptable trade-off for an academic pilot's traffic volume but would not be sufficient at production scale.

12.4 Performance Efficiency

Decisions: Lambda scales automatically per request with no capacity planning; DynamoDB on-demand similarly avoids capacity planning. The jobId-matchScore-index GSI lets the recruiter ranking query run as a single indexed Query instead of a full table Scan with client-side sorting.

Trade-off: Lambda cold starts (Node.js 20 runtime) add roughly 300-450 ms of latency on the first request after idle, observed directly on serveFrontend during testing. We accepted this rather than paying for Provisioned Concurrency, since the pilot's traffic pattern — bursty around demo and career-fair events, otherwise idle — makes a constantly-warm Lambda poor value for the actual access pattern.

12.5 Cost Optimization

Decisions: Every compute and data service in the stack is pay-per-use: Lambda, API Gateway HTTP API, DynamoDB on-demand, S3, and Step Functions Standard. There is no idle, always-on infrastructure billed by the hour.

Trade-off: The Hugging Face Inference API bills per call/token outside the AWS account; at higher volume, a reserved-throughput or self-hosted model could be cheaper per resume than pay-as-you-go API calls. We chose Hugging Face specifically because Amazon Bedrock — which would let us reason about LLM cost inside the same AWS bill and the same cost-allocation tags — is blocked in this Learner Lab account. This is a constrained-environment trade-off, not a free architectural choice.

12.6 Sustainability

Decisions: Serverless and managed services (Lambda, DynamoDB on-demand, S3, API Gateway) shift hardware-utilization efficiency to AWS, which operates at a scale and utilization rate no project-level always-on server could match. Nothing in this stack runs when there is no request to serve.

Trade-off: Every resume analysis makes a network round-trip to an external LLM provider (Hugging Face) rather than an in-region inference endpoint, adding cross-network data transfer that a co-located Bedrock call would avoid. As with Cost Optimization above, this is a direct consequence of Bedrock being blocked in this Learner Lab account rather than a deliberate sustainability choice, and would be revisited first if this project moved to a production AWS account.

13. Challenges and Solutions

Challenge	Solution
Learner Lab 4-hour credential TTL	Documented refresh process; GitHub Secrets updatable via UI
HF API token format in Secrets Manager	Stored as JSON {token:hf_...}; Lambda uses JSON.parse().token
Terraform local-exec on Windows and Ubuntu	Changed interpreter from bash to pwsh (PowerShell Core)
LLM response not always pure JSON	Regex extraction of first braces block; fallback error handling
Lambda caching old invalid token	Forced cold start by updating CACHE_BUST environment variable

14. Future Work

- Resume parsing improvements: structured section extraction using Textract AnalyzeDocument API.
- Self-hosted LLM: deploy Qwen2.5-7B on SageMaker for lower latency and no external API dependency.
- Resume improvement loop: allow students to receive revised resume suggestions and re-analyze iteratively.
- CloudFront CDN: add distribution layer for sub-100ms frontend load times globally.
- Multi-region deployment: active-active across us-east-1 and us-west-2 for higher availability.

15. Conclusion

The AI Resume Analyzer and Job Matcher successfully demonstrates the power of serverless cloud-native architecture for AI-powered applications. By combining AWS managed services with the Hugging Face Inference API, the system delivers sub-90-second resume analysis with zero server management overhead. The fully Terraform-managed infrastructure and GitHub Actions CI/CD pipeline ensure reproducibility and quality. The 71-test suite provides confidence in code correctness, while CloudWatch alarms ensure operational visibility.

16. References

1. Amazon Web Services. (2024). AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/
2. Amazon Web Services. (2024). AWS Step Functions Developer Guide. https://docs.aws.amazon.com/step-functions/
3. Amazon Web Services. (2024). Amazon Textract Developer Guide. https://docs.aws.amazon.com/textract/
4. Qwen Team. (2024). Qwen2.5: A Party of Foundation Models. arXiv:2412.15115.
5. HashiCorp. (2024). Terraform Documentation. https://developer.hashicorp.com/terraform
6. GitHub. (2024). GitHub Actions Documentation. https://docs.github.com/en/actions